

I develop UniversalToolchain/Wist2, a modular .NET platform for compiling and executing DSLs with Bytecode/AIR, a reference interpreter and CIL/DynamicMethod code generation. I implemented an experimental verifier-gated AIR → SSA → AIR route and optimization passes. A separate PS-form Analyzer demonstrates conservative program analysis and exact-reference testing.

AIR → SSA → AIR IR transformations and optimization passes guarded by structural verifiers.	1,325 tests · 75 .NET projects Verified baseline dated July 14, 2026: 0 warnings and 0 errors.	1st place · 104/104 Conservative memory-dependence analysis across loop iterations.
---	--	---

EXPERIENCE

- 2024 — present** **UniversalToolchain / Wist2 — Creator and Primary Developer**
Compiler/runtime platform · .NET · public alpha
 - Designed Source → AST → Bytecode → AIR → Backend → Execution, a reference interpreter and CIL/DynamicMethod code generation.
 - Implemented AIR → SSA → AIR, SCCP-lite, constant and branch folding, unreachable-code elimination and dead pure-instruction elimination.
 - SSA is verified after conversion and every pass; AIR before the backend; supported semantics are covered by interpreter/CIL equivalence checks.
- July-August 2026** **MCST — Compiler Engineering Intern**
LLVM track · C++23 · remote
 - Developing graph-analysis components over a shared data model with explicit contracts.
 - Maintaining CMake, warnings-as-errors, ASan/UBSan, clang-tidy, explicit include dependencies and reproducible I/O tests.
- 2021 — present** **Programming Tutor and Mentor**
C/C++ · C# · Python · algorithms
 - Taught about 50 students; I diagnose gaps, explain execution models and review code.

SELECTED PROJECTS

- Compiler/runtime platform · .NET**
UniversalToolchain / Wist2
A multi-IR execution pipeline, modular composition, capability contracts, structural verifiers and two execution paths.
- Program analysis · C17**
PS-form Memory Dependence Analyzer
Address normalization, range/residue/GCD filters, exact affine reasoning, bounded search and an exact reference model.
- Educational lab · Rust / x86-64**
Codegen & Register Allocation Playground
An SSA-like IR, liveness, linear scan and simulated annealing, x86-64 generation through iced-x86 and interpreter/native comparison.
- Algorithm library · C++23**
AdvancedAlgorithms
12 reusable graph, tree, string and data-structure modules with differential tests, invariants and ASan/UBSan.

EXPERTISE

- Architecture and IR**
Lexing and parsing, AST, Bytecode and AIR; a reference interpreter and CIL/DynamicMethod code generation; modular composition and explicit capability contracts.
- Analysis and optimization**
CFG and SSA; AIR-to-SSA lowering and SSA-to-AIR emission; SCCP, constant and branch folding, unreachable-block and dead pure-instruction elimination, liveness and memory-dependence analysis.
- Correctness**
Structural IR verification, interpreter/backend equivalence checks, exact reference models, differential and metamorphic tests, and negative cases.
- Languages and toolchain**
C#/.NET, C++23, C17 and Python; CMake, GitHub Actions, ASan/UBSan and clang-tidy.

RECOGNITION

- Prize-winner of the HSE “Vysshaya Proba” Olympiad in competitive and industrial programming; prize-winner of the regional Russian Informatics Olympiad.
- Two highest-tier diplomas in the MEPhI Junior competition and a highest-tier Baltic Science and Engineering Competition diploma for compiler/runtime projects.

EDUCATION

HSE University student